

Day 15: タスク編集・削除を実装しよう

今日のゴール

既存のタスクを編集・削除できる機能を実装します。変更内容を保存し、不要なタスクを削除できるようにします。

【スクリーンショット: タスク編集画面】

なぜこれを作るのか?

タスク情報は常に変化します。**タスク編集は文章の推敲のようなもの**。一度書いた文章でも、後から読み返して修正することで、より正確で分かりやすくなります。同様に、タスクも状況に応じて内容を更新することで、チームの認識を統一できます。

実装ステップ一覧

ステップ	作業内容	所要時間
Step 1	編集ページを作成	10分
Step 2	既存データの取得・表示	15分
Step 3	更新処理の実装	15分
Step 4	削除処理の実装	10分

合計時間: 約50分

Step 1: 編集ページを作成（10分）

実装:

```
// filepath: src/app/projects/[projectId]/tasks/[taskId]/
'use client';

import { useParams } from 'next/navigation';
import { Box, Typography } from '@mui/material';

export default function EditTaskPage() {
  const params = useParams();
  const taskId = params.taskId as string;

  return (
    <Box sx={{ p: 3 }}>
      <Typography variant="h4">タスク編集</Typography>
    </Box>
  );
}
```

確認ポイント: /projects/[projectId]/tasks/[taskId]/editにアクセスできる

Step 2: 既存データの取得・表示（15分）

実装:

```

// filepath: src/app/projects/[projectId]/tasks/[taskId]/
import { useState, useEffect } from 'react';
import { api } from '@trpc/react';
import {
  TextField,
  Button,
  Select,
  MenuItem,
  FormControl,
  InputLabel,
  CircularProgress,
} from '@mui/material';

export default function EditTaskPage() {
  const params = useParams();
  const taskId = params.taskId as string;

  const { data: task, isLoading } = api.task.getById.useQuery(
    { taskId },
    { enabled: !!taskId }
  );

  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');
  const [priority, setPriority] = useState('MEDIUM');
  const [dueDate, setDueDate] = useState('');

  useEffect(() => {
    if (task) {
      setTitle(task.title);
      setDescription(task.description || '');
      setPriority(task.priority);
      setDueDate(
        task.dueDate
          ? new Date(task.dueDate).toISOString().split('T')[0]
          : ''
      );
    }
  }, [task]);

```

```

    if (isLoading) {
      return <CircularProgress />;
    }

    return (
      <Box sx={{ p: 3, maxWidth: 600 }}>
        <Typography variant="h4" sx={{ mb: 3 }}>タスク編集</Typography>
        <TextField
          fullWidth
          required
          label="タイトル"
          value={title}
          onChange={(e) => setTitle(e.target.value)}
          sx={{ mb: 2 }}
        />
        { /* 他のフィールドも同様 */ }
      </Box>
    );
  }
}

```

確認ポイント: タスクの既存データがフォームに表示される

Step 3: 更新処理の実装 (15分)

実装:

```
// filepath: src/app/projects/[projectId]/tasks/[taskId]/
import { useRouter } from 'next/navigation';

export default function EditTaskPage() {
  const params = useParams();
  const taskId = params.taskId as string;
  const projectId = params.projectId as string;
  const router = useRouter();

  const updateMutation = api.task.update.useMutation({
    onSuccess: () => {
      router.push(`/projects/${projectId}/tasks`);
    },
  });

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    updateMutation.mutate({
      id: taskId,
      title,
      description,
      priority,
      dueDate: dueDate ? new Date(dueDate) : undefined,
    });
  };

  return (
    <Box sx={{ p: 3, maxWidth: 600 }}>
      <form onSubmit={handleSubmit}>
        {/* フィールド */}
        <Button
          type="submit"
          variant="contained"
          fullWidth
          disabled={updateMutation.isPending}
        >
```

```
        {updateMutation.isPending ? '更新中...' : '更新'}  
      </Button>  
    </form>  
  </Box>  
);  
}
```

確認ポイント: タスクが更新され、一覧ページに戻る

Step 4: 削除処理の実装（10分）

実装:

```
// filepath: src/app/projects/[projectId]/tasks/[taskId]/
export default function EditTaskPage() {
  const deleteMutation = api.task.delete.useMutation({
    onSuccess: () => {
      router.push(`/projects/${projectId}/tasks`);
    },
  });

  const handleDelete = () => {
    if (confirm('このタスクを削除しますか?')) {
      deleteMutation.mutate({ id: taskId });
    }
  };

  return (
    <Box sx={{ p: 3, maxWidth: 600 }}>
      <form onSubmit={handleSubmit}>
        {/* フィールド */}

        <Box sx={{ display: 'flex', gap: 2, mt: 2 }}>
          <Button
            type="submit"
            variant="contained"
            fullWidth
            disabled={updateMutation.isPending}
          >
            更新
          </Button>
          <Button
            variant="outlined"
            color="error"
            fullWidth
            onClick={handleDelete}
            disabled={deleteMutation.isPending}
          >
            削除
          </Button>
        </Box>
      </form>
    </Box>
  );
}
```



```
        </Button>
      </Box>
    </form>
  </Box>
);
}
```

確認ポイント: 確認ダイアログが表示され、タスクが削除される

学んだこと

- **useEffect:** データ取得後にstateを初期化するタイミング制御
- **日付のフォーマット:** ISOString から YYYY-MM-DD 形式への変換
- **confirm関数:** ブラウザ標準の確認ダイアログ
- **ボタン配置:** display: flex と gap で横並びレイアウト
- **color="error":** MUIの赤色ボタン（削除操作に適した色）

今日のまとめ

- ☐ タスク編集ページを作成できた
- ☐ 既存データをフォームに表示できた
- ☐ 更新処理を実装できた
- ☐ 削除処理と確認ダイアログを実装できた

⚠ つまづきポイント

問題	原因	解決策
フォームが空のまま	useEffect の依存配列に task がない	依存配列に [task] を追加
日付がずれる	タイムゾーンの考慮不足	split('T')[0] で日付部分のみ取得
削除が即実行される	confirm の戻り値をチェックしていない	if (confirm(...)) で確認

次回予告

Day 16では、タスクのステータス変更とタイマー機能を実装します。